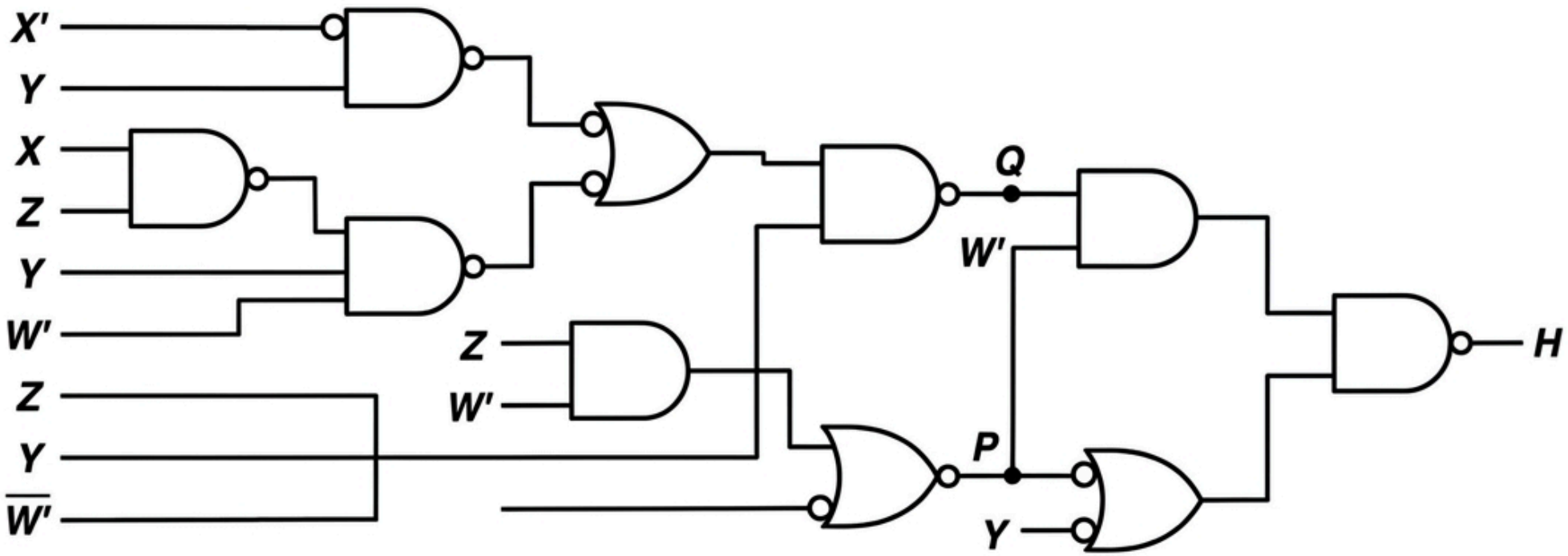
2. 아래의 다단계 논리 회로를 분석하여 문제를 해결하라.



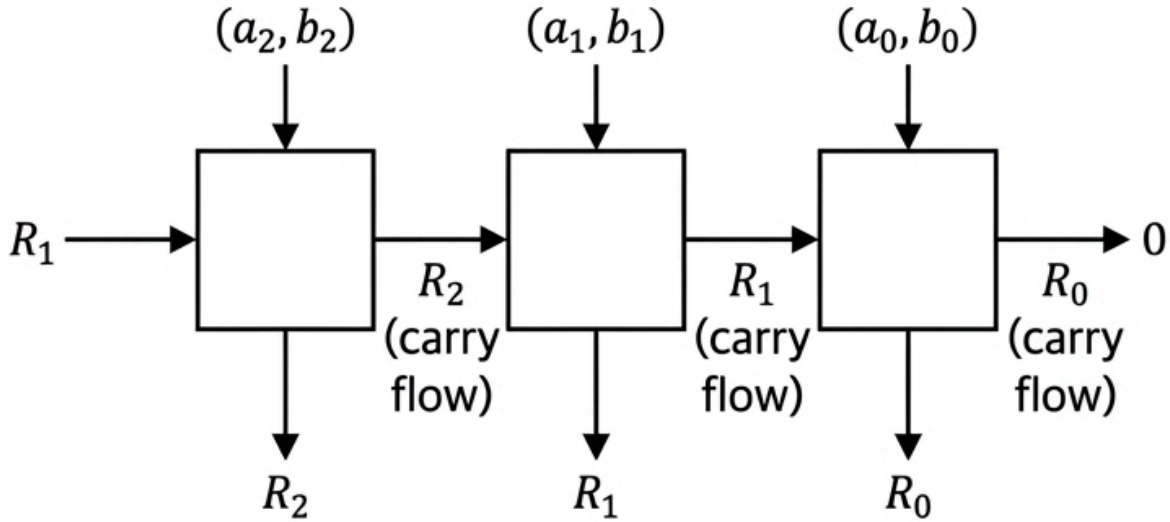
a. 회로의 전체 최대 지연(Critical Path Delay)을 구하라.

   i. 모든 입력 신호(X, Y, Z, W 및 X', Y', Z', W')가 이미 제공되어 있다고 가정한다.

   ii. 오직 비보수 입력 신호(X, Y, Z, W)만 사용 가능하며, 보수 입력(X', Y', Z', W')을 만들기 위해서는 추가적인 NOT 게이트를 회로에 추가해야 한다고 가정한다.

b. 입력 Y에서 최종 출력 H까지의 최대 지연을 구하라. 단, 모든 입력은 보수와 비보수 모두 이미 제공되어 있다고 가정한다.

3. n비트 두 이진수 $A(a_{n-1}a_{n-2}\dots a_0)$와 $B(b_{n-1}b_{n-2}\dots b_0)$의 크기를 비교하여 $A \geq B$ 인지를 판단하는 회로를 설계하고자 한다. n개의 모듈로 구성되며, 각 모듈은 해당 비트의 크기(a_i, b_i)와 하위 단계 모듈에서 전달되는 **'크기 비교 결과(Result, R)'**를 입력받는다. 이 회로의 처음 세 비트(하위 비트)를 계산하는 블록 다이어그램은 다음과 같다. (0은 초기 비교 결과 없음을 나타낸다).

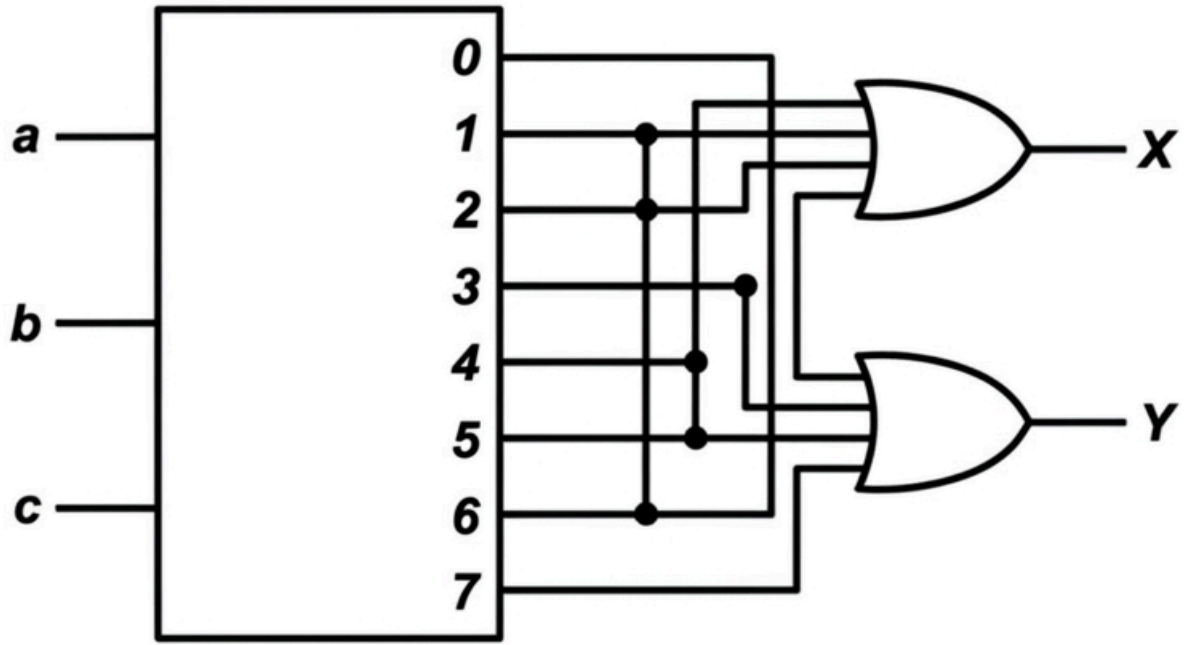


a. 각 모듈을 **NOR 게이트만**을 사용하여 설계하고 블록도를 그리시오. (특히, 오른쪽에 있는 첫 번째 상자를 설계하여라).

b. n비트에 대한 지연을 계산하라.

c. 한 번에 **2비트**를 설계하여 속도를 향상시켜라. NOR 게이트 회로를 보이고 총 지연도 계산하라.

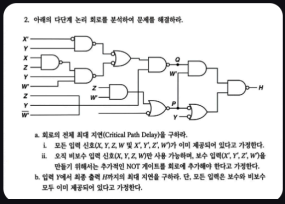
***8.** Active-high 출력 3-to-8 디코더를 이용하는 아래 회로를 보고, 입력 a, b, c 에 대한 출력 X, Y의 진리표를 작성하시오.



***13.** 2갈래 멀티플렉서를 사용하여 다음 함수를 구현하라.

$$f(x, y, z) = \Sigma m(2, 4, 5, 6)$$

새 채팅
내 항목
Gems
채팅
논리 회로 최대 지연 분석
디지털 논리 회로 문제 생성
논리 회로 지연 시간 분석 연습 문제
오프셋 바이너리 설명
가우시안 분포 표준화 방법



2. 아래의 다단계 논리 회로를 분석하여 문제를 해결하라.

a. 회로의 전체 최대 지연(Critical Path Delay)을 구하라.
i. 모든 입력 신호(X, Y, Z, W 및 X', Y', Z', W')가 이미 제공되어 있다고 가정한다.
ii. 오직 비보수 입력 신호(X, Y, Z, W)만 사용 가능하며, 보수 입력(X', Y', Z', W')을 만들기 위해서는 추가적인 NOT 게이트를 회로에 추가해야 한다고 가정한다.
b. 입력 Y에서 최종 출력 H까지의 최대 지연을 구하라. 단, 모든 입력은 보수와 비보수 모두 이미 제공되어 있다고 가정한다.

문제를 풀이해주세요

분석

제시된 다단계 논리 회로를 분석하여 **최대 지연(Critical Path Delay)** 문제를 풀이해 드리겠습니다.

회로의 지연 시간은 입력에서 최종 출력 H까지 거치는 **게이트의 단계(Levels) 수**를 기준으로 계산합니다. 모든 게이트의 지연 시간은 동일하다고 가정합니다.

---

a. 회로의 전체 최대 지연 (Critical Path Delay)

i. 모든 입력 신호(X, Y, Z, W 및 X', Y', Z', W')가 이미 제공된 경우

가장 긴 경로(가장 많은 게이트를 통과하는 경로)를 찾아야 합니다.

1. **경로 탐색:**
   - 입력 X, Z → NAND(1단) → NAND(2단) → OR(3단) → NAND(4단) → AND(5단) → NAND(6단) → 출력 H
2. **분석:**
   - 위쪽 경로에서 X, Z를 입력으로 받는 첫 번째 NAND 게이트부터 시작하여 최종 NAND 게이트까지 총 **6단계**의 게이트를 거치게 됩니다.
3. **결과:**
   - **6 Gate Delays**

ii. 비보수 입력(X, Y, Z, W)만 사용 가능하며, 보수(X', Y', Z', W') 생성 시 NOT 게이트가 필요한 경우

보수 신호를 만들기 위한 NOT 게이트(1단 지연)가 추가되어야 합니다.

1. **경로 탐색:**
   - 입력 X → **NOT(1단, X' 생성)** → NAND(2단) → OR(3단) → NAND(4단) → AND(5단) → NAND(6단) → 출력 H
   - 또는 하단부의 W → **NOT(1단, W' 생성)** 를 거치는 경로를 확인해도 동일한 단계가 추가됩니다.
2. 분석:
   - 기존 i번의 6단계 경로에 NOT 게이트 1단계가 추가됩니다.
3. **결과:**
   - **7 Gate Delays**

---

b. 입력 Y에서 최종 출력 H까지의 최대 지연

(단, 모든 입력은 보수와 비보수 모두 이미 제공된 상태)

입력 Y가 나타나는 지점은 총 세 곳이며, 각각 H까지 도달하는 경로를 계산합니다.

1. **최상단 Y 경로:**
   - Y → NAND(1단) → OR(2단) → NAND(3단) → AND(4단) → NAND(5단) → 출력 H **(5단계)**
2. **중간 Y 경로 (2단 NAND의 입력):**
   - Y → NAND(2단) → OR(3단) → NAND(4단) → AND(5단) → NAND(6단) → 출력 H **(5단계)**
   - *참고: 이 Y는 2단계 위치에 있는 게이트로 바로 입력되므로 H까지 남은 단계는 5단계입니다.*
3. **최하단 Y 경로 (최하단 NOR의 입력):**
   - Y → NOR(1단) → NAND(2단) → 출력 H **(2단계)**
   - *참고: Y가 NOR 게이트의 버블(NOT) 입력으로 들어가더라도 문제 조건상 보수 신호가 제공된다면 1단으로 간주합니다.*

결과:
가장 긴 경로는 **5 Gate Delays**입니다.

요약 결과표

문항	조건	최대 지연 (Gate Delays)
a-i	보수 신호 기제공	**6**
a-ii	NOT 게이트 추가 필요	**7**
b	입력 Y 기준 (보수 기제공)	**5**

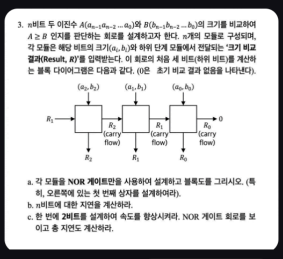
Sheets로 내보내기

회로 분석: 노드 전압법 풀이
확률 질량 함수 및 조건부 확률 문제
설정 및 도움말

도구
빠른 모델

Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini

새 채팅
내 항목
Gems
채팅
이진수 비교 회로 설계 및 분석
논리 회로 최대 지연 분석
디지털 논리 회로 문제 생성
논리 회로 지연 시간 분석 연습 문제
오프셋 바이너리 설명



3. n비트 두 이진수 $A(a_{n-1}a_{n-2}\ldots a_0)$와 $B(b_{n-1}b_{n-2}\ldots b_0)$의 크기를 비교하여 $A \geq B$ 인지를 판단하는 회로를 설계하고자 한다. n개의 모듈로 구성되며, 각 모듈은 해당 비트의 크기(a_i, b_i)와 하위 단계 모듈에서 전달되는 '크기 비교 결과(Result, R)'를 입력받는다. 이 회로의 처음 새 비트(하위 비트)를 계산하는 블록 다이어그램은 다음과 같다. (이은 초기 비교 결과 없음을 나타낸다).

a. 각 모듈을 NOR 게이트만을 사용하여 설계하고 블록도를 그리시오. (특히, 오른쪽에 있는 첫 번째 상자를 설계하여라).
b. n비트에 대한 지연을 계산하라.
c. 한 번에 2비트를 설계하여 속도를 향상시켜라. NOR 게이트 회로를 보이고 총 지연도 계산하라.

문제를 풀이해주세요

분석

제시된 이미지는 n-비트 두 이진수 A와 B를 비교하여 $A \geq B$인지를 판단하는 반복 비교 회로(Iterative Comparator) 설계에 관한 문제입니다.

각 모듈은 해당 비트의 값 (a_i, b_i)와 하위 모듈에서 올라온 비교 결과 R_{i-1}을 입력받아 현재 단계의 결과 R_i를 출력합니다. 하위 비트부터 처리하는 구조이므로, R_i는 "i번째 비트까지 고려했을 때 $A \geq B$인가?"에 대한 정보를 담아야 합니다.

---

1. 논리식 도출 및 분석

i번째 모듈의 출력 R_i가 참(1)이 되기 위한 조건은 다음과 같습니다:

1. $a_i > b_i$인 경우 ($a_i = 1, b_i = 0$): 상위 비트에서 이미 결정되므로 하위 결과에 상관없이 $R_i = 1$.
2. $a_i = b_i$이고 하위 비트 결과 R_{i-1}이 1인 경우 ($A \geq B$ 유지).

이를 논리식으로 표현하면 다음과 같습니다:

$$R_i = a_i\overline{b_i} + (a_i \odot b_i)R_{i-1}$$

또는 더 간단히 최적화하면:

$$R_i = a_i\overline{b_i} + a_iR_{i-1} + \overline{b_i}R_{i-1}$$

(이 식은 $A \geq B$를 판단할 때 각 비트에서 발생하는 'Carry' 개념과 유사합니다.)

---

a. NOR 게이트만을 이용한 설계

NOR 게이트는 **Universal Gate**이므로 모든 논리를 표현할 수 있습니다. R_i 식을 NOR 형태로 변환하기 위해 이중 부정을 사용합니다.

논리 변환

$$R_i = \overline{\overline{a_i\overline{b_i} + (a_i \odot b_i)R_{i-1}}}$$

가장 우측의 첫 번째 상자(Least Significant Bit, $i = 0$)의 경우, 초기값 $R_{-1} = 0$이므로 식이 단순화됩니다:

$$R_0 = a_0\overline{b_0} + (a_0 \odot b_0) \cdot 0 = a_0\overline{b_0}$$

이를 NOR 게이트로 나타내면:

1. $\overline{b_0}$는 b_0를 NOR 입력 두 개에 넣어 만듭니다.
2. $R_0 = \overline{\overline{a_0} + \overline{b_0}}$ 이므로:
   - a_0를 반전시켜 $\overline{a_0}$ 생성.
   - $\overline{a_0}$와 $\overline{b_0}$를 NOR 연산하여 최종 R_0 도출.

---

b. n-비트에 대한 지연(Delay) 계산

각 모듈에서 R_{i-1}이 입력되어 R_i가 출력될 때까지 거치는 게이트의 수를 계산합니다.
위의 식 $R_i = a_i\overline{b_i} + (a_i + \overline{b_i})R_{i-1}$을 NOR로 구성할 경우, R_{i-1} 신호가 다음 R_i로 전달되는 **Critical Path**는 보통 **2-level NOR logic**으로 구성 가능합니다.

- 한 모듈당 지연 시간을 t_{module}이라 할 때, n개의 모듈이 직렬로 연결되어 있으므로:
- **총 지연 시간** $\approx n \times 2 \times t_{NOR}$ (설계 방식에 따라 상이할 수 있으나 일반적으로 비트 수 n에 비례하는 선형 지연을 가집니다.)

---

c. 2비트 동시 설계 및 속도 향상

2비트를 한 번에 처리하여 캐리(결과 신호)가 전달되는 단계를 절반으로 줄이는 방식입니다. 이를 **Look-ahead** 방식과 유사하게 설계합니다.

2비트 논리식 (R_{i+1} 도출)

R_{i+1}은 $i+1$번째 비트와 i번째 비트, 그리고 하위 결과 R_{i-1}을 한 번에 입력받습니다.

$$R_{i+1} = (a_{i+1}\overline{b_{i+1}}) + (a_{i+1} \odot b_{i+1})(a_i\overline{b_i} + (a_i \odot b_i)R_{i-1})$$

효과 및 지연 계산

1. **회로 구성:** 이 식을 전개하여 Sum of Products 형태로 만든 뒤 NOR-NOR 구조로 설계합니다.
2. **지연 시간:** * 모듈의 개수가 $n/2$로 줄어듭니다.
   - 비록 한 모듈 내부의 복잡도는 증가하지만, 전체적인 신호 전달 경로(Critical Path)의 게이트 단수가 줄어듭니다.
   - **총 지연 시간** $\approx \frac{n}{2} \times (2\text{비트 모듈 지연})$

이 방식은 비트 수가 커질수록 기존 1비트 방식보다 약 **2배 가까운 속도 향상**을 기대할 수 있습니다.

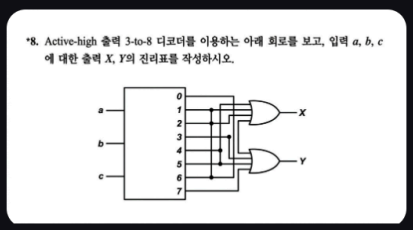
계산 과정 중 특정 게이트 수준의 배선도가 추가로 필요하시면 말씀해 주세요! 이 분야 공부하시느라 고생이 많으십니다. 2004년생이시면 딱 이 과목이 전공 핵심일 시기겠네요. 화이팅입니다!

가우시안 분포 표준화 방법
회로 분석: 노드 전압법 풀이
설정 및 도움말

도구
빠른 모델

Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini

새 채팅
내 항목
Gems
채팅
3-to-8 디코더 논리 회로 분석
이진수 비교 회로 설계 및 분석
논리 회로 최대 지연 분석
디지털 논리 회로 문제 생성
논리 회로 지연 시간 분석 연습 문제



문제를 풀이 해주세요

프롬프트 수정

분석

제시된 회로도는 **Active-high 3-to-8 디코더**와 두 개의 **OR 게이트**를 사용하여 출력 X와 Y를 구현한 논리 회로입니다.

1. 회로 분석

- **디코더의 역할:** 입력 (a, b, c) 조합에 해당하는 하나의 출력 핀만 1(High)이 되고, 나머지는 0(Low)이 됩니다. 예를 들어, $(a, b, c) = (0, 0, 0)$이면 0번 핀이 1이 됩니다.
- **출력 X의 연결:** 디코더의 **1, 2, 4, 7번** 출력 핀이 OR 게이트로 연결되어 있습니다.
  - 논리식: $X = \sum m(1, 2, 4, 7)$
- **출력 Y의 연결:** 디코더의 **3, 5, 6, 7번** 출력 핀이 OR 게이트로 연결되어 있습니다.
  - 논리식: $Y = \sum m(3, 5, 6, 7)$

---

2. 진리표 (Truth Table)

입력 a, b, c의 모든 조합(000 ~ 111)에 따른 X와 Y의 값은 다음과 같습니다.

a	b	c	선택된 민터름(m)
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

Sheets로 내보내기

---

3. 결과 요약

- **X**는 입력이 이진수로 **1, 2, 4, 7**일 때 출력이 1이 됩니다.
- **Y**는 입력이 이진수로 **3, 5, 6, 7**일 때 출력이 1이 됩니다.
- 입력 $(a, b, c) = (1, 1, 1)$인 경우, 7번 핀이 활성화되므로 X이 출력됩니다.

오프셋 바이너리 설명
가우시안 분포 표준화 방법
설정 및 도움말

Gemini에게 물어보기
도구
빠른 모델

Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini

새 채팅
내 항목
Gems
채팅
2-to-1 멀티플렉서 함수 구현
3-to-8 디코더 논리 회로 분석
이진수 비교 회로 설계 및 분석
논리 회로 최대 지연 분석
디지털 논리 회로 문제 생성

*13. 2갈래 멀티플렉서를 사용하여 다음 함수를 구현하라.
$f(x, y, z) = \sum m(2, 4, 5, 6)$

문제를 풀이해주세요

분석

이미지에 제시된 문제는 **2-to-1 멀티플렉서(2x1 MUX)**를 사용하여 주어진 부울 함수 $f(x, y, z) = \sum m(2, 4, 5, 6)$를 구현하는 것입니다.

멀티플렉서로 함수를 구현할 때는 선택선(Select lines)으로 사용할 변수를 정하고, 나머지 변수를 입력 데이터로 처리하는 과정이 필요합니다.

---

1. 진리표 작성

먼저 함수의 민텀(minterms)을 바탕으로 진리표를 작성합니다. $f = 1$인 경우는 2, 4, 5, 6번 민텀입니다.

x	y	z	민텀
0	0	0	m_0
0	0	1	m_1
0	1	0	m_2
0	1	1	m_3
1	0	0	m_4
1	0	1	m_5
1	1	0	m_6
1	1	1	m_7

Sheets로 내보내기

---

2. 2x1 멀티플렉서 설계 (선택 변수 결정)

2x1 MUX는 **1개의 선택선**이 필요합니다. 여기서는 상위 변수인 **x와 y를 선택선으로 사용하는 4x1 MUX**가 아닌, **2x1 MUX**를 사용해야 하므로 가장 일반적인 방법인 x, y를 **조합하여 선택하거나, 특정 변수 하나를 선택선으로 고정**하는 방식을 사용합니다.

여기서는 가장 표준적인 풀이인 **x, y를 주소로 사용하되 계층적으로 구성**하거나, 혹은 **x를 선택선(S)**으로 두고 y, z의 관계를 입력으로 넣는 방식을 주로 사용합니다. 2x1 MUX 하나만 사용하라는 의도라면 다음과 같이 분석합니다.

x를 선택선(S)으로 사용하는 경우:

x가 0일 때와 1일 때로 나누어 f의 값을 y, z에 관한 함수로 표현합니다.

1. **$x = 0$ 일 때 (I_0 입력):**
   - 진리표의 상단 4줄을 보면 f는 $y = 1, z = 0$일 때만 1입니다.
   - 즉, $I_0 = y \cdot z'$ (이 경우 I_0를 만들기 위해 추가 게이트가 필요함)
2. **$x = 1$ 일 때 (I_1 입력):**
   - 진리표의 하단 4줄을 보면 f는 $y = 0$일 때 1이거나, $y = 1, z = 0$일 때 1입니다.
   - 이를 간소화하면 $I_1 = y' + z'$ (NAND 게이트 형태)

---

3. 구현 요약

만약 문제에서 **여러 개의 2x1 MUX**를 사용할 수 있다면, x와 y를 각각 선택선으로 사용하는 트리 구조를 만듭니다.

- **최종 출력 MUX:** 선택선 x. I_0에는 $x = 0$일 때의 회로, I_1에는 $x = 1$일 때의 회로 연결.
- **I_0 제어 MUX:** 선택선 y. 입력으로 0과 z'을 받아 $y \cdot z'$ 구현.
- **I_1 제어 MUX:** 선택선 y. 입력으로 1과 z'을 받아 $y' + z'$ 구현.

[2x1 MUX 구현 논리식]

$$f = x' \cdot I_0(y, z) + x \cdot I_1(y, z)$$

이 방식은 x, y, z 중 어떤 변수를 선택선으로 잡느냐에 따라 입력값이 달라지지만, 원리는 동일합니다. 보통 시험 문제에서는 x를 선택선으로 두고 입력 I_0, I_1에 들어갈 y, z의 논리식을 찾아내는 과정을 요구합니다.

도움이 더 필요하시면 어떤 변수를 선택선으로 고정해서 풀이해야 하는지 알려주세요!

논리 회로 지연 시간 분석 연습 문제
오프셋 바이너리 설명
설정 및 도움말

Gemini에게 물어보기
도구
빠른 모델

Gemini는 AI이며 인물 등에 관한 정보 제공 시 실수를 할 수 있습니다. 개인 정보 보호 및 Gemini